

北京信息科技大学

毕业设计（论文）

题目：基于 LangGraph 的多 Agent 协作框架在医疗审计场景的设计与实现

学院：计算机学院

专业：计算机科学与技术

学生姓名：程浩然 班级/学号 计科 2401-本/2024070013

指导老师/督导老师：乔文豹

起止时间：2026 年 2 月 23 日至 2026 年 5 月 22 日

摘 要

针对个人体检报告、检验单和病历摘要等医疗资料来源分散、格式不统一、处理过程缺少追溯的问题，本文设计并实现了一套基于 LangGraph 的多 Agent 协作医疗审计辅助系统。系统采用前后端分离架构，后端以 FastAPI、SQLAlchemy 和 MySQL 为基础，围绕原始文件、OCR 结果、文档版本、结构化指标、任务事件和审计事件建立数据模型；同时通过 Provider 抽象接入 OCR、标准化和大语言模型能力。综合审计报告模块以 LangGraph 状态图组织文档质量审计、时间线构建、指标一致性检查、风险提示、证据绑定、冲突复核、合规检查、报告生成和安全审查等节点，通过条件路由和回环机制保证审计结论能够补充证据、回退复核并持久化。前端实现文档接入、文档库、指标探索、智能洞察和综合审计报告展示，能够直观看到审计节点与边的流转。测试结果表明，系统能够基于 Mock 体检数据完成从资料入库到报告生成的闭环。

关键词：LangGraph；多 Agent；医疗审计；OCR；结构化入库；证据追溯；FastAPI

Abstract

This thesis designs and implements a LangGraph-based multi-agent collaboration framework for medical audit scenarios. The system targets personal health documents such as physical examination reports, laboratory reports and clinical notes. It builds a traceable data pipeline covering upload, OCR, normalization, versioned storage, metric querying, audit orchestration and frontend visualization. The backend is implemented with FastAPI, SQLAlchemy and MySQL, and the external OCR, normalization and LLM capabilities are accessed through replaceable providers. The audit report module uses LangGraph as a state machine and coordinates document quality checking, timeline construction, metric consistency review, risk analysis, evidence binding, conflict review, compliance checking, report composition and safety review. Conditional routing and cyclic transitions make the workflow auditable and suitable for iterative evidence completion. The frontend presents document management, metric exploration, intelligent insight and an audit report workspace with visible node and edge transitions. Tests based on mock health examination data show that the system can complete an end-to-end workflow from document ingestion to final audit report generation.

Keywords: LangGraph; multi-agent; medical audit; OCR; normalization; evidence traceability; FastAPI

目 录

摘要（中文）.....	
（英文）.....	
第一章 绪论.....	1
1.1 实现背景和意义.....	1
1.2 国内外实现现状.....	2
1.3 本文主要工作.....	3
第二章 相关技术与系统基础.....	4
2.1 医疗资料标准化与数据治理.....	4
2.2 Provider 抽象与外部能力接入.....	5
2.3 LangGraph 状态机与多 Agent 协作.....	6
2.4 本项目已有实现基础.....	7
第三章 需求分析与总体设计.....	8
第四章 系统详细实现.....	14
第五章 测试与运行效果分析.....	24
结束语.....	29
参考文献.....	30

第一章 绪论

本章首先说明课题的实现背景和意义，随后概述国内外相关技术实现现状，最后说明本文围绕基于 LangGraph 的多 Agent 协作框架所完成的主要工程实现工作。

1.1 实现背景和意义

随着医院信息化、个人健康档案和体检服务的普及，普通用户能够获得的医疗资料越来越多。这些资料既包括体检总检报告、检验单、影像检查结论，也包括病历摘要、随访记录和健康管理建议。资料数量的增加并没有自动带来更清晰的健康信息组织，相反，不同机构的报告模板、指标命名、单位表达和异常标记存在差异，用户在长期管理资料时往往只能依赖人工翻阅，难以快速判断同一指标在不同时间点的变化，也难以知道某个提示结论来自哪一份原始报告。

生成式大语言模型在医疗问答、文本总结和临床知识组织方面表现出较强的语言理解能力，国内外文献已经讨论了其在医学场景中的机遇、限制和安全边界[1-4]。但医疗场景具有高敏感性和高责任属性，仅依赖一次性生成式回答无法满足审计要求。系统必须清楚记录数据来源、处理步骤、证据绑定、异常依据和人工复核边界，使每一条提示都能回到具体文档和具体字段，而不是停留在无法解释的自然语言结论。

国家医院信息化建设和电子病历应用评价均强调数据标准化、质量控制、过程留痕和信息安全[5-6]，个人信息安全规范也要求在个人敏感信息处理过程中落实最小必要、访问控制和可追溯措施[7]。因此，本课题并不把系统实现目标设定为替代医生诊断，而是定位为医疗资料整理和医疗审计辅助：通过工程化方式把原始资料转化为结构化数据，再使用状态机式多 Agent 协作框架完成质量检查、风险提示、证据补全和报告输出。

从工程实现角度看，传统线性工作流可以完成固定步骤的数据处理，但难以表达“发现证据不足后回到证据节点”“安全审查不通过后回到报告生成节点”“引用检查不通过后回到审计节点”等循环过程。LangGraph 提供了状态图、条件边和持久化状态的实现基础，适合把医疗审计过程组织为可回环、可观测、可恢复的状态机。本文围绕这一特点完成系统设计，使课题题目中的“基于 LangGraph 的多 Agent 协作框架”在系统核心流程中得到明确体现。

图 1-1 给出了本文系统面向的医疗审计场景与系统边界。图中将输入资料、处理流程、审计输出和非诊断性边界分开表示，强调系统输出是健康资料审计和风险提示，而不是临床诊断。

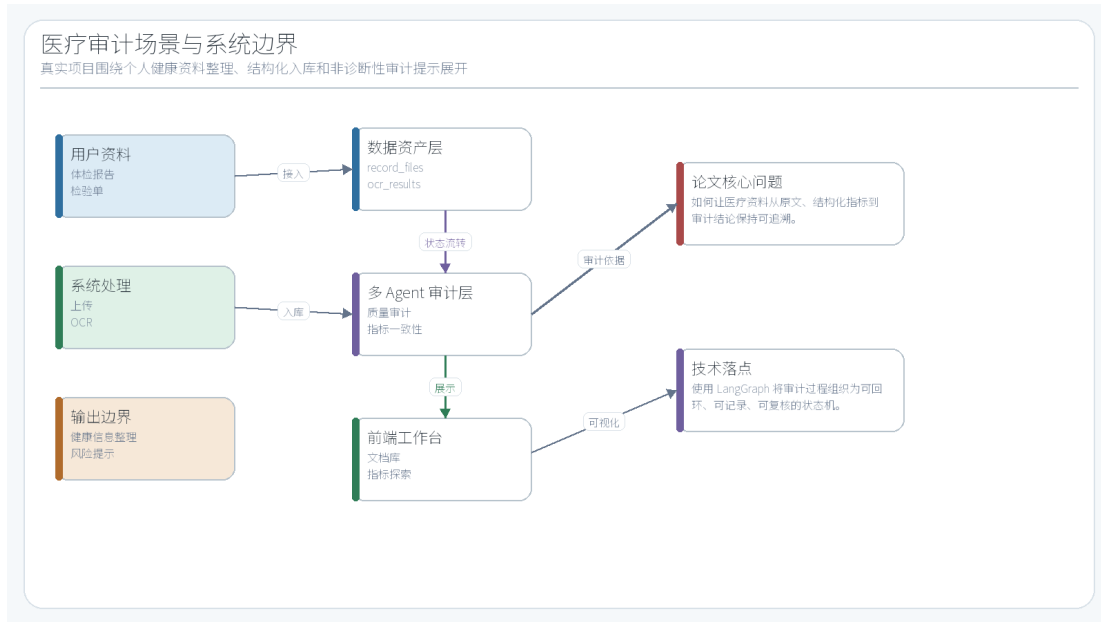


图 1-1 医疗审计场景与系统边界

1.2 国内外实现现状

在医疗人工智能方向，国外以 Med-PaLM、GPT 类模型和临床知识评测为代表，重点关注医学知识问答、临床文本理解和辅助解释能力。Nature Medicine 和 Nature 等期刊的相关工作说明，大语言模型已经能够编码一定临床知识，但其输出仍然受到数据来源、提示方式、事实一致性和安全边界影响[11-12]。国内相关文献也指出，医疗大模型落地需要结合真实业务流程、数据治理和责任边界，而不是简单将通用问答模型搬到医疗场景中[1-4]。

在工程架构方向，检索增强生成、推理与工具调用结合、多 Agent 协作成为提升生成式系统可控性的常用方式。RAG 通过外部知识或业务数据补充模型上下文，降低脱离事实回答的概率[8]；ReAct 将推理轨迹和行动调用交替组织，使系统能够根据环境反馈调整任务计划[9]；AutoGen 等多 Agent 框架则强调用不同角色承担拆解、执行、评审和修正职责[10]。这些工作为本文拆分审计节点、保存中间状态和设计回环复核提供了参考。

在软件工程实现方向，Web 系统需要稳定的数据模型、清晰的接口边界和可测试的服务层。FastAPI 提供类型化接口和自动文档能力[14]，SQLAlchemy 提供 ORM 与事务管理能力[15]，Pydantic 提供数据校验和序列化能力[16]。本文系统在后端使用这些组件构建资料处理和审计接口，在智能能力部分通过 Provider 抽象将 OCR、标准化和 LLM 调用从业务流程中隔离出来，从而降低替换外部服务时对业务代码的影响。

与单纯聊天机器人或固定工作流不同，本文实现重点在于把医疗资料处理、结构化入库和多 Agent 审计串成闭环。系统不仅要回答“生成了什么报告”，还要说明“报告从哪些文档来、经过哪些节点、每个节点输出什么、为什么可以结束或为什么需要回退”。这也是开题报告中“多 Agent 协作、任务审计、来源可追溯和前端展示”的延续。后续章节将围绕数据模型、接口设计和 LangGraph 状态机实现展开。

1.3 本文主要工作

本文完成的主要工作包括四个方面。第一，设计并实现医疗资料数据底座，覆盖用户、健康记录、原始文件、OCR 结果、结构化文档、文档版本、指标、后台任务、任务事件、Provider 调用事件和审计报告运行记录等对象。所有关键对象均有明确字段和关联关系，能够支持后续论文中的 ER 图、字段表和接口说明。

第二，设计并实现资料处理流程。系统支持文件上传、文本型 OCR 抽取、LLM/规则混合标准化、指标入库、版本快照和指标查询。标准化结果不直接覆盖原文，而是同时保留 OCR 原文、标准化 payload、版本 hash 和结构化指标，从数据结构上支持回溯和复核。

第三，设计并实现基于 LangGraph 的综合审计报告生成流程。该流程不是一条直线，而是包含 `audit_router` 和 `final_router` 两个路由节点，并允许 Agent 节点完成后回到 `audit_router`；当引用检查或安全审查未通过时，`final_router` 能够回到审计或报告生成节点。该设计体现了状态机、条件边、回环复核和过程持久化。

第四，完成前端模块和测试数据准备。前端包括文档接入、文档库、指标探索、智能洞察和综合审计报告模块；Mock 数据包括多份体检、化验、病历和影像报告，用于支撑端到端测试与论文图表展示。本文使用真实源码、数据库模型和测试脚本生成图表，避免图表与项目实现脱节。

第二章 相关技术与系统基础

本章介绍系统实现所依赖的医疗资料标准化、Provider 抽象、LangGraph 状态机和项目已有基础，重点说明这些技术如何服务于医疗审计场景中的可追溯、可复核和可展示要求。

2.1 医疗资料标准化与数据治理

医疗资料的结构化不是简单的文本摘要。以体检和检验报告为例，同一项指标可能出现中文名、英文缩写、不同单位、不同参考范围和不同异常标记；病历摘要中又包含症状、既往史、用药史、检查建议等叙事性事实。若系统只保存自然语言回答，后续无法完成趋势查询、异常复核和证据定位。因此本文将标准化入库作为系统基础能力，而不是把它作为前端展示的附属功能。

本文采用“原始文件、OCR 结果、当前文档投影、文档版本、结构化指标”五层数据结构。原始文件保存用户上传时的文件名、显示名、类型、大小和字节内容；OCR 结果保存识别文本、原始 payload、provider 名称和修订号；结构化文档保存当前可查询的文档类型、类别、报告日期和标准化 payload；文档版本保存每次标准化快照及 hash；指标表保存可用于搜索和时序分析的 `name`、`value_text`、`value_numeric`、`unit` 和 `observed_at`。

这种设计的意义在于将审计问题拆成可验证的数据问题。若报告结论提到某项血糖异常，系统可以通过 `evidence_items` 回到 `measurements.value_numeric`，再通过 `document_version_id` 回到文档版本和 OCR 原文；若标准化算法发生变化，旧版本仍可保留，新的标准化结果以新版本形式写入，避免覆盖历史证据。该方式与医院信息化建设强调的数据质量、过程留痕和复核要求保持一致[5-7]。

2.2 Provider 抽象与外部能力接入

表 2-1 Provider 抽象及当前配置

抽象接口	当前配置	功能说明
OCRProvider	plaintext	负责从文件字节中抽取文本，当前演示环境使用 plaintext，保留 baidu_ocr 和 openai_compatible_vision 扩展点。
NormalizationProvider	llm_direct	负责把 OCR 原文转换为文档类型、报告日期、指标数组和叙事事实；当前为 llm_direct，并带规则兜底。
LLMProvider	openai_compatible	负责对话、洞察和部分标准化能力，当前通过 openai_compatible 接入兼容模型服务。
StorageProvider	database_inline	负责文件存储，当前使用 database_inline，便于本地演示和测试复现。

Provider 抽象的核心作用是隔离业务流程与外部能力。OCR、LLM 和存储服务具有明显的不稳定性和环境差异：本地演示时可能只处理纯文本，生产部署时可能需要百度 OCR、视觉模型或对象存储；标准化能力也可能在规则解析、LLM 解析和混合解析之间切换。如果业务服务直接依赖某个具体 SDK，后续替换成本会很高，也不利于记录调用耗时、错误类别和重试结果。

图 2-1 展示了本文系统的 Provider 接入方式。业务服务只依赖统一接口，ProviderGateway 负责记录 provider_events、映射异常、统计耗时并隐藏外部服务差异。论文中的测试环境读取 .env 后得到当前配置：ocr_provider 为 plaintext，normalization_provider 为 llm_direct，llm_provider 为 openai_compatible，storage_provider 为 database_inline。该图由项目配置和 Provider 代码生成，反映当前真实工程状态。

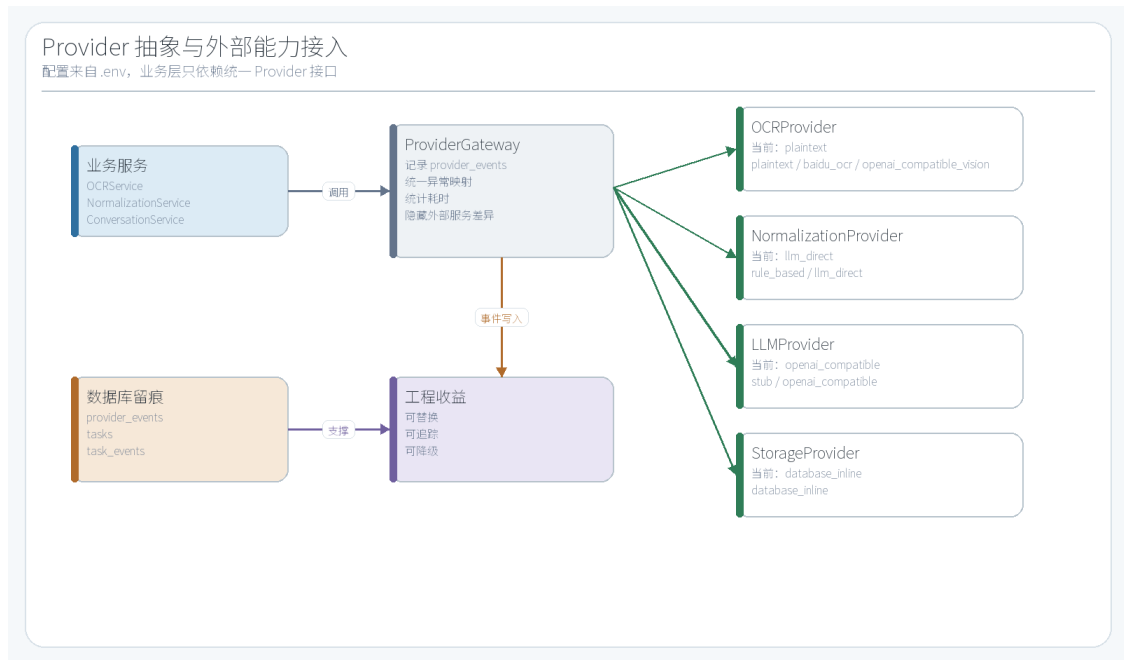


图 2-1 Provider 抽象与外部能力接入

2.3 LangGraph 状态机与多 Agent 协作

LangGraph 是面向 Agent 和复杂工作流的状态图框架，适合表达包含状态传递、条件路由、循环和持久化的执行过程[13]。与普通顺序工作流相比，状态图的关键优势在于节点之间并不只是固定下一步，而是可以根据共享状态选择不同边；节点执行结果会写回 GraphState，后续节点基于同一状态继续判断。这种机制非常适合医疗审计场景，因为审计过程经常出现证据不足、结论需要复核、报告需要重写和安全审查不通过等情况。

本文将多 Agent 理解为多个具有明确职责的审计节点，而不是简单堆叠多个聊天机器人。每个节点只负责一个有限任务，例如文档质量检查、时间线构建、指标一致性检查、风险提示、证据绑定、冲突复核、合规检查、报告生成、引用检查和安全审查。节点之间通过 AuditGraphState 共享数据，路由节点根据 completed_agents、citation_issues、safety_issues、needs_report_revision 等字段决定下一步。

图 2-2 说明了本文使用 LangGraph 的基本机制。START 进入状态加载节点后，audit_router 按状态分派多个审计 Agent；普通 Agent 完成后回到 audit_router；报告链路完成后进入 final_router；final_router 根据引用检查和安全审查结果决定回到 audit_router、回到 report_composer 或进入 persist_report。该机制使系统具备状态机和回环能力，也使前端可以根据事件流展示节点高亮和边流转。

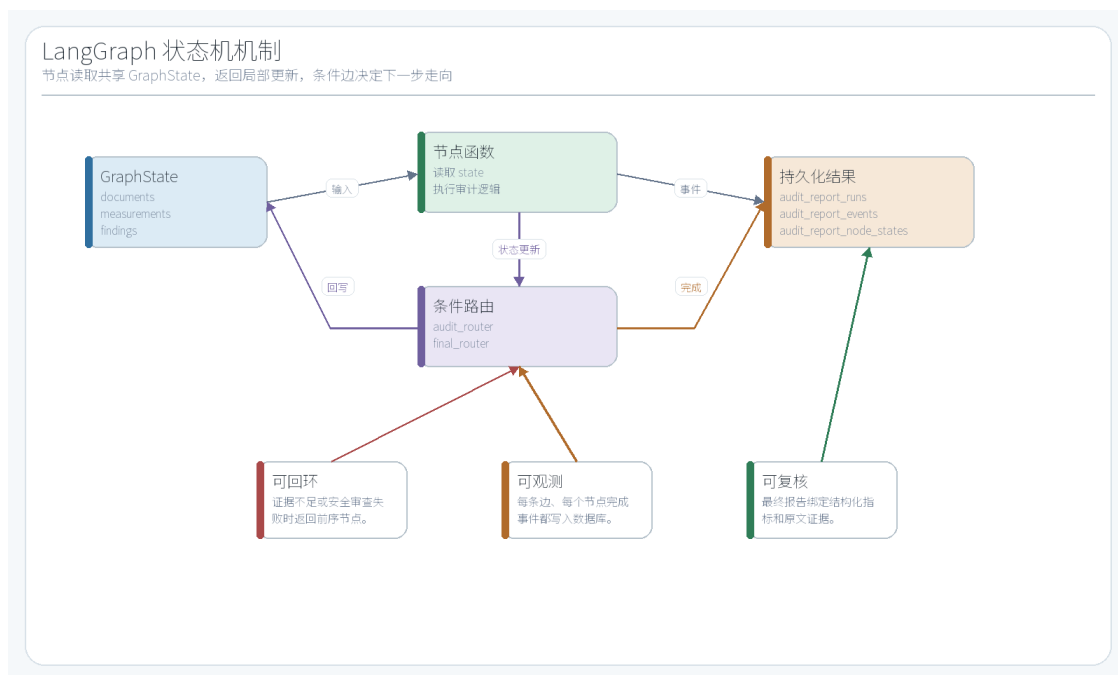


图 2-2 LangGraph 状态机机制

2.4 本项目已有实现基础

本文实现不是从空白项目开始，而是在开题报告确定的工程路线基础上完成系统化整理和强化。已有基础包括 FastAPI 后端、SQLAlchemy 模型、MySQL 数据库连接、用户认证、文件上传、OCR 结果保存、标准化服务、指标查询、智能洞察对话、综合审计报告接口、React 前端页面和 Mock 体检数据脚本。中期阶段暴露出的主要问题是系统展示没有充分体现 LangGraph 架构，因此本文后续实现重点转向综合审计报告模块和状态机可视化。

在当前版本中，后端路由统一挂载到 /api，已包含 auth、files、ocr、ingestion、documents、document-versions、records、measurements、query、tasks、insight、chat 和 audit-reports 等分组。数据库模型覆盖核心业务对象，测试脚本能够创建 exam@healthdoc.local 测试账户并写入 15 份 Mock 文档、65 条结构化指标和 14 条叙事事实。综合审计报告模块已经能够持久化 audit_report_runs、audit_report_events 和 audit_report_node_states。

因此，本文后续章节的图表和表格均以当前源码为依据生成。需要特别说明的是，当前实现中的审计节点以规则审计和证据绑定为主，LLM 主要用于标准化和洞察能力；本文不会虚假描述为每个节点都由大模型自主生成，而是准确表述为“LangGraph 编排多个审计节点，结合规则审计、证据绑定、条件路由和 LLM 标准化/洞察能力”。这种表述更符合项目真实状态，也更符合工程类毕业设计的要求。

第三章 需求分析与总体设计

本章从功能需求、总体架构、业务流程、后端接口、数据库结构和安全追溯等方面进行总体设计，说明系统如何将文档处理、结构化入库和综合审计报告生成组织为完整工程闭环。

3.1 需求分析

表 3-1 系统功能需求与数据支撑

需求	说明	支撑对象
资料接入	用户能够上传体检报告、检验单、病历摘要和影像文本结论，系统保存原始文件和文件元数据。	record_files
OCR 与标准化	系统能够抽取文本并生成结构化文档、版本快照和指标记录。	ocr_results、 extracted_documents、 document_versions、 measurements
指标查询	用户能够按名称、时间和文档查询结构化指标，用于后续审计和趋势展示。	measurements API
综合审计	系统能够选择多个文档版本，通过 LangGraph 多节点流程生成综合审计报告。	audit_report_runs、events、 node_states
过程可视化	前端能够展示审计节点、边流转、最终报告和历史记录。	audit-reports API
安全边界	系统输出为非诊断性风险提示，所有结论必须保留来源或提示人工复核。	evidence_items、 safety_reviewer

系统的用户角色为个人健康资料管理者。用户使用系统的基本过程为：登录账号，上传资料，触发 OCR 或文本抽取，执行标准化入库，在文档库中查看文档版本和指标，在综合审计报告模块中选择若干报告生成审计报告。由于本课题面向医疗审计辅助而非诊断，系统必须在输出层明确边界，避免出现“确诊”“治愈”“必须立即用药”等不适合审计场景的表述。

非功能需求主要包括可追溯性、可扩展性、稳定性和可测试性。可追溯性要求关键数据从原始文件到最终报告都有路径；可扩展性要求 OCR、标准化和 LLM 服务可替换；稳定性要求外部服务失败时有规则兜底或明确错误状态；可测试性要求接口、数据库和 LangGraph 审计流程能在 Mock 数据下复现。本文设计的数据库、Provider 抽象和 audit_report_events 正是围绕这些非功能需求展开。

3.2 系统总体架构

系统采用前后端分离和分层后端架构。前端负责用户交互、文件选择、模块切换、数据展示和审计图流转；后端提供统一 REST 接口，按照 API 层、服务层、Provider 层和数据访问层组织；数据库保存业务数据、任务事件和审计状态；外部能力通过 Provider 层接入。综合

审计报告模块位于服务层核心位置，负责把文档版本和指标数据组织为 LangGraph 初始状态，再将每个节点输出写入审计事件表。

图 3-1 展示系统总体架构。与开题报告中的技术路线一致，该架构从资料接入开始，经过 OCR、结构化抽取、版本化存储和指标查询，最终进入多 Agent 审计与前端展示。图中把数据库和 Provider 能力放在后端服务的支撑层，说明系统不是单一前端页面，也不是单一大模型调用，而是一个包含数据闭环、服务编排和审计留痕的工程系统。

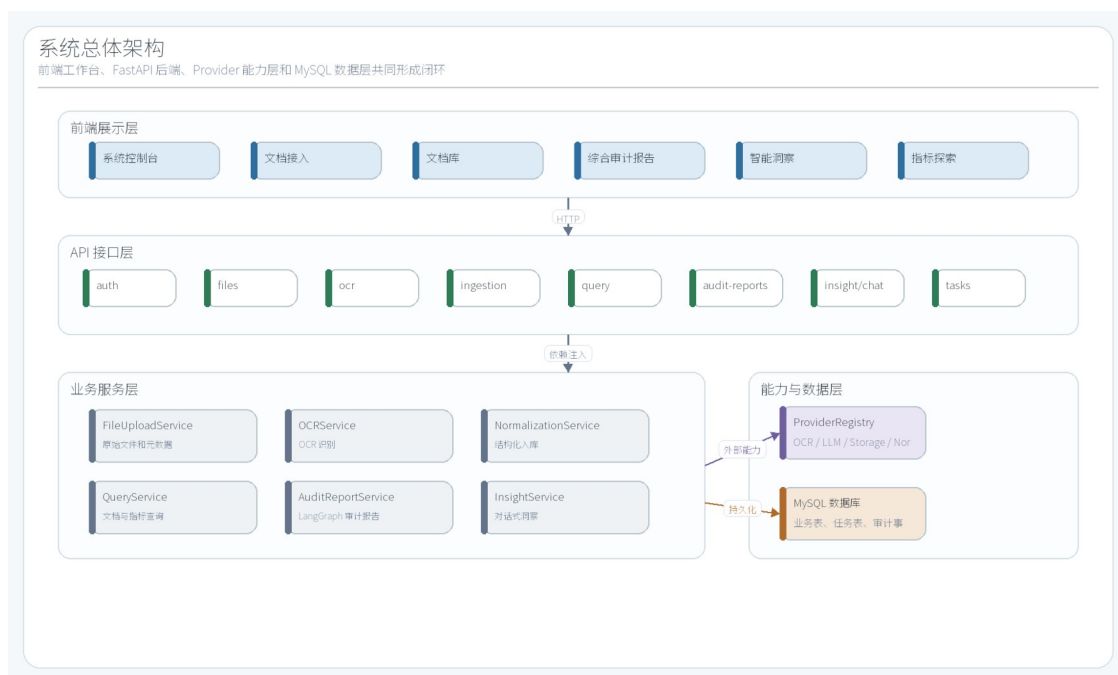


图 3-1 系统总体架构

3.3 业务流程设计

业务流程围绕“上传资料后形成可审计数据资产”展开。用户上传文件后，系统创建 records 和 record_files，随后通过 OCR 接口生成 ocr_results。标准化接口读取 OCR 原文，将文档类型、报告日期、结构化指标和叙事事实写入 extracted_documents，同时创建 document_versions 和 measurements。综合审计报告模块再读取选中的文档版本和指标，启动 LangGraph 状态机，持续产生事件和节点状态，最后持久化 final_report。

图 3-2 给出了医疗资料处理与审计业务流程。流程图强调两个闭环：第一个闭环是从原始文件到结构化指标的入库闭环，第二个闭环是从文档版本到综合审计报告的状态机闭环。前者保证系统有真实数据可审计，后者保证审计过程不是黑盒生成。



图 3-2 医疗资料处理与审计业务流程

3.4 后端分层与接口设计

表 3-2 主要 API 分组

接口分组	数量	典型接口
auth	3	POST /auth/register; POST /auth/login; GET /auth/me
files	3	POST /files/upload; GET /files/{file_id}/documents; GET /files/{file_id}/measurements
ocr	5	POST /ocr/files/{record_file_id}/extract; GET /ocr/files/{record_file_id}/revisions; GET /ocr/files/{record_file_id}/revisions/current; ...
ingestion	1	POST /ingestion/ocr-results/{ocr_result_id}/normalize
documents	7	GET /documents; GET /documents/{document_id}; GET /documents/{document_id}/ver

		sions; ...
measurements	3	GET /measurements; GET /measurements/search; GET /measurements/timeseries
tasks	6	GET /tasks; GET /tasks/{task_id}; GET /tasks/{task_id}/events; ...
audit-reports	6	POST /audit-reports; POST /audit-reports/{run_id}/execute; GET /audit-reports; ...
insight	6	GET /insight/sessions; GET /insight/sessions/{session_id}; GET /insight/sessions/{session_id}/messages; ...
chat	10	POST /chat/conversations; GET /chat/conversations; GET /chat/conversations/{conversation_id}; ...

后端分层结构如图 3-3 所示。API 层负责认证、参数解析和响应模型；服务层负责业务编排，例如文件上传、OCR 任务、标准化入库和综合审计报告；Provider 层封装外部能力；模型层由 SQLAlchemy ORM 定义数据库结构。这样的分层避免了接口函数直接操作复杂业务流程，也使论文中的每个功能点都能定位到具体代码层。

API 设计遵循资源化思路。files 分组负责文件上传和文件关联数据查询；ocr 分组负责 OCR 修订记录；ingestion 分组负责将 OCR 结果标准化；documents 和 document-versions 分组负责文档与版本查询；measurements 分组负责指标搜索和时序；tasks 分组负责后台任务状态和事件；audit-reports 分组负责综合审计报告创建、执行、事件轮询和节点状态查询。表 3-2 中的接口数量来自当前 FastAPI APIRouter 快照。

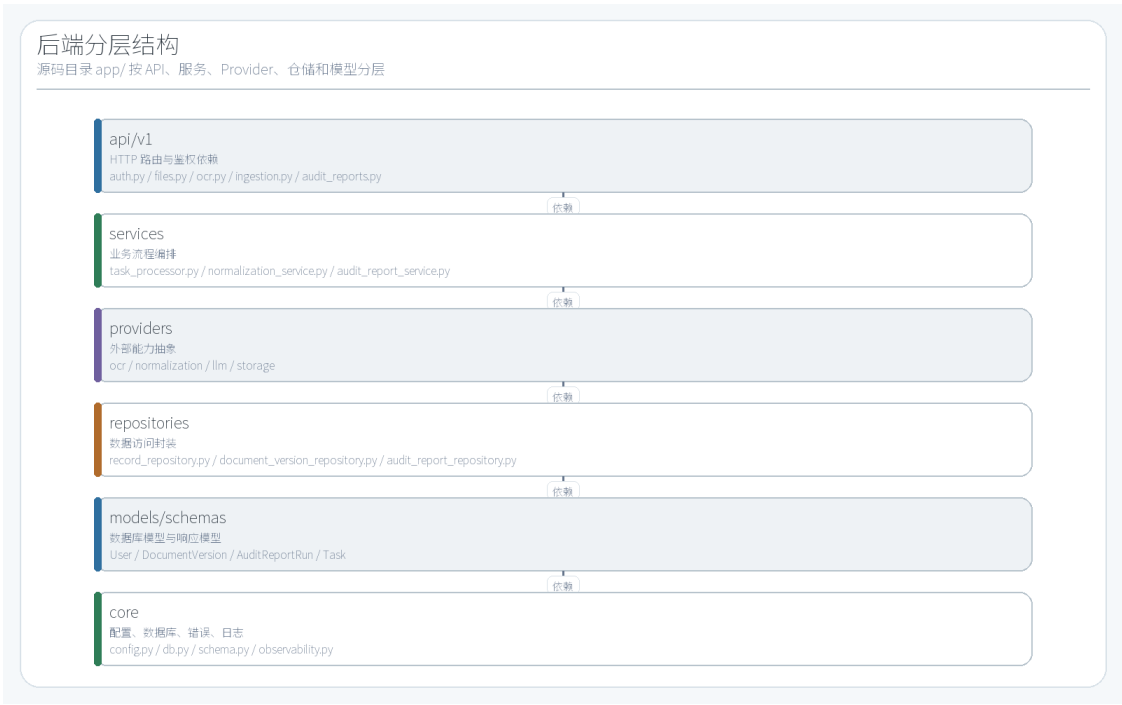


图 3-3 后端分层结构

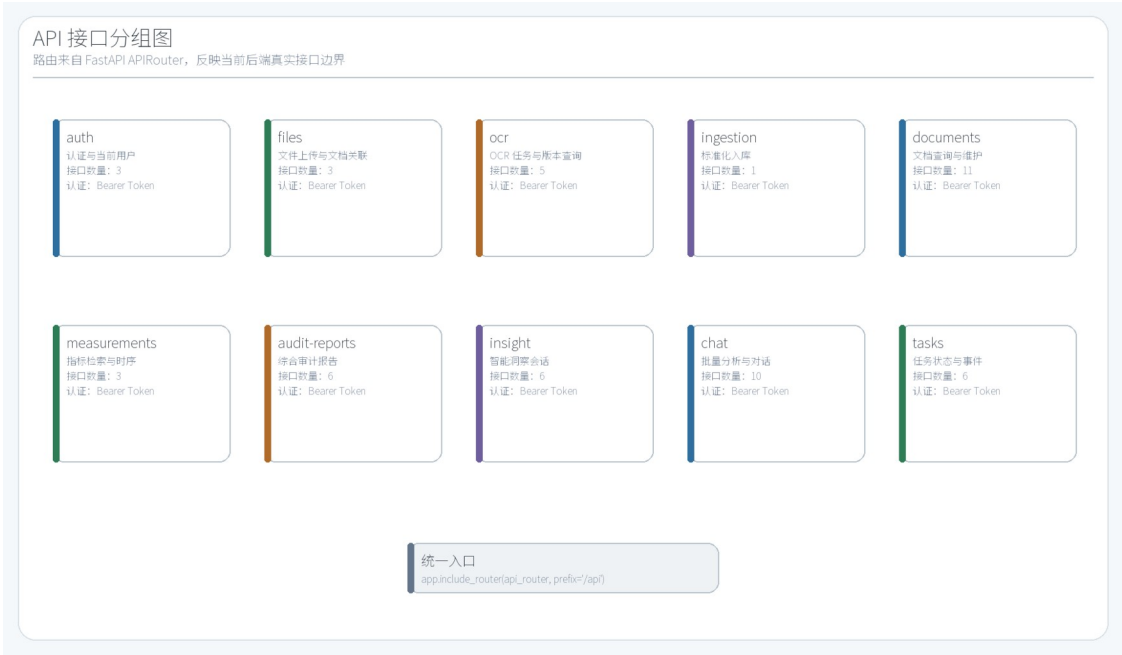


图 3-5 API 接口分组图

3.5 数据库设计

表 3-3 核心数据表职责

数据表	职责	主要字段
users	用户账号	id, email, password_hash, created_at
record_files	原始文件与文件字节	id, record_id, original_filename,

		display_name, content_type, size_bytes, content_bytes, storage_provider, storage_key, created_at
ocr_results	OCR 识别结果与修订	id, record_file_id, revision_number, supersedes_ocr_result_id, is_current, provider_name, status, raw_text, raw_payload, created_at
extracted_documents	当前标准化文档投影	id, ocr_result_id, current_ocr_result_id, record_id, record_file_id, document_type, document_category, display_name, status, report_date, normalized_payload, created_at
document_versions	标准化文档版本快照	id, document_id, version_number, supersedes_version_id, is_current, created_from_ocr_result_id, snapshot_hash, report_date, normalized_payload, created_at
measurements	结构化指标索引	id, extracted_document_id, document_version_id, name, value_text, value_numeric, unit, observed_at, created_at
audit_report_runs	综合审计报告运行	id, user_id, title, status, selected_document_version_id s, graph_state, final_report, iteration_count...

数据库设计是系统可追溯能力的基础。本文将资料处理主链路、异步任务链路和审计报告状态机链路分开建模。资料处理主链路从 users 到 records、record_files、ocr_results、extracted_documents、document_versions 和 measurements；任务链路由 tasks、task_events 和 provider_events 记录；审计链路由 audit_report_runs、audit_report_events 和 audit_report_node_states 记录。三条链路通过 user_id、record_id、document_version_id、task_id 和 run_id 建立关联。

图 3-4 为核心数据库 ER 图。图中字段来自 SQLAlchemy 模型快照，保留主键、外键和关键业务字段。由于 tasks 和 audit_report_runs 字段较多，图中展示论文分析所需字段，完整字段清单放在附录 A。该设计能够支持从最终报告向前追溯到审计事件、节点输出、文档版本、OCR 原文和原始文件。

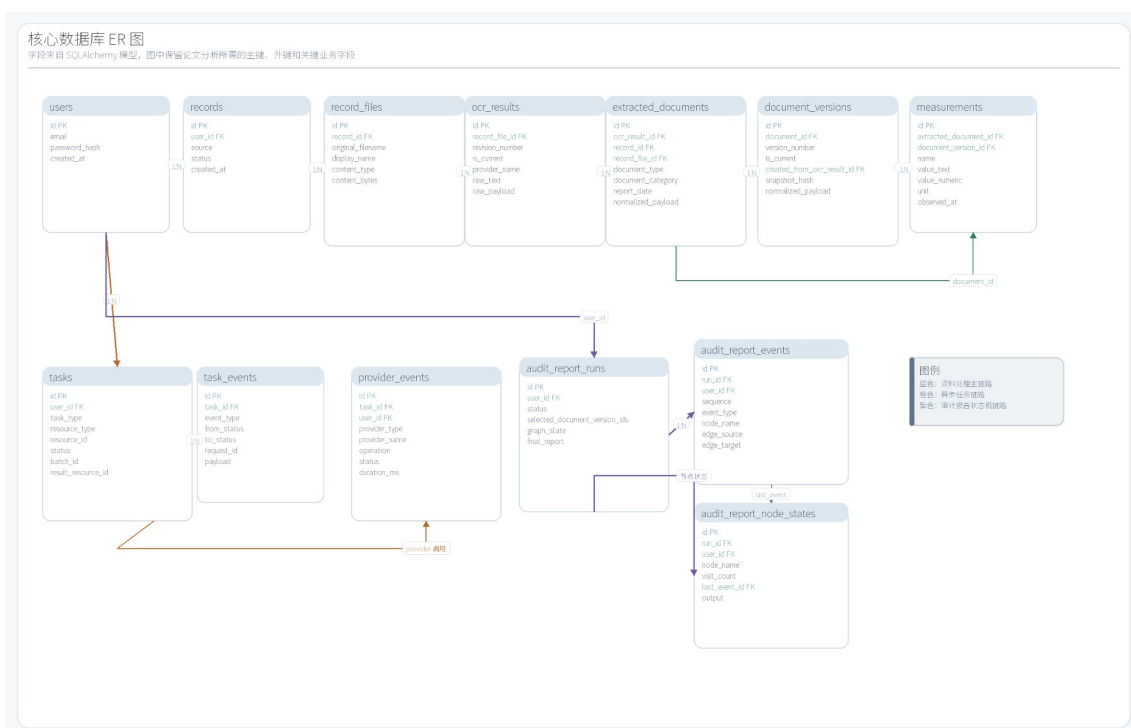


图 3-4 核心数据库 ER 图

3.6 安全与可追溯设计

医疗资料通常包含敏感个人信息，系统设计必须避免无边界扩散。本文在数据层通过 user_id 进行用户隔离，业务查询均围绕当前用户过滤；在文件和 OCR 层保留原文与 provider 信息；在任务层记录 task_events 和 provider_events；在审计层记录 audit_report_events 和 audit_report_node_states；在输出层通过 safety_reviewer 检查不适合审计场景的医疗措辞。

图 3-6 展示安全与可追溯设计。其重点不是做复杂权限体系，而是在毕业设计原型系统范围内保证每次资料处理、Provider 调用和审计报告生成都有可查记录。对于医疗审计辅助系统而言，可追溯性本身也是安全边界的一部分，因为无法追溯来源的提示不应进入最终报告。

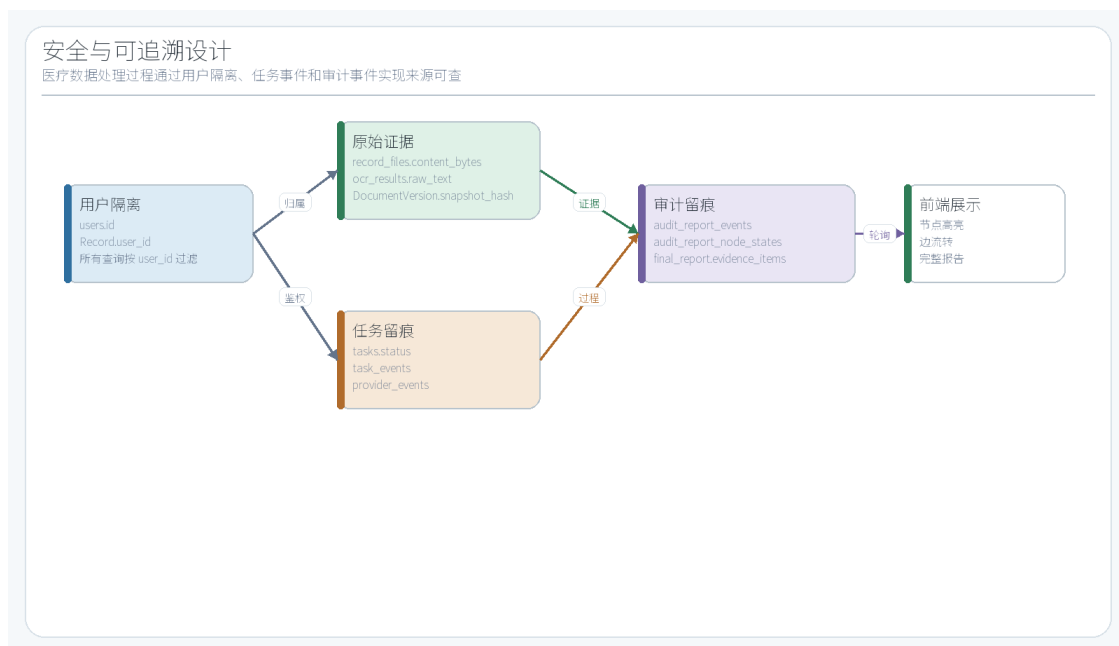


图 3-6 安全与可追溯设计

第四章 系统详细实现

本章说明文件上传与 OCR、标准化入库、LangGraph 审计状态机、审计事件持久化和前端模块的具体实现，重点体现系统中真实数据如何在后端状态图和前端流程图之间保持一致。

4.1 文件上传与 OCR 处理实现

文件上传流程由前端提交文件，后端 files API 校验并写入 records 和 record_files。record_files 不仅保存 original_filename 和 display_name，也保存 content_type、size_bytes、storage_provider、storage_key 和 content_bytes。当前演示环境采用 database_inline 存储，便于本地复现和测试；若后续接入对象存储，只需要替换 StorageProvider。

OCR 处理通过 ocr API 触发。系统创建或更新 tasks，并将具体抽取动作交给 OCRProvider。当前 Provider 为 plaintext，适合处理 Mock 文本报告和本地演示；保留的 baidu_ocr 和 openai_compatible_vision 扩展点可用于真实图片或 PDF 场景。处理完成后，系统写入 ocr_results，字段包括 record_file_id、revision_number、supersedes_ocr_result_id、is_current、provider_name、status、raw_text、raw_payload 和 created_at。

图 4-1 展示文件上传与 OCR 处理时序。该图从前端、files API、FileUploadService、数据库、ocr API、TaskProcessor 和 OCRProvider 之间的调用关系出发，说明系统把长耗时任务和外部能力调用从同步页面操作中拆开。即使当前演示 Provider 是 plaintext，任务和事件结构仍然为后续真实 OCR 服务保留了工程扩展空间。

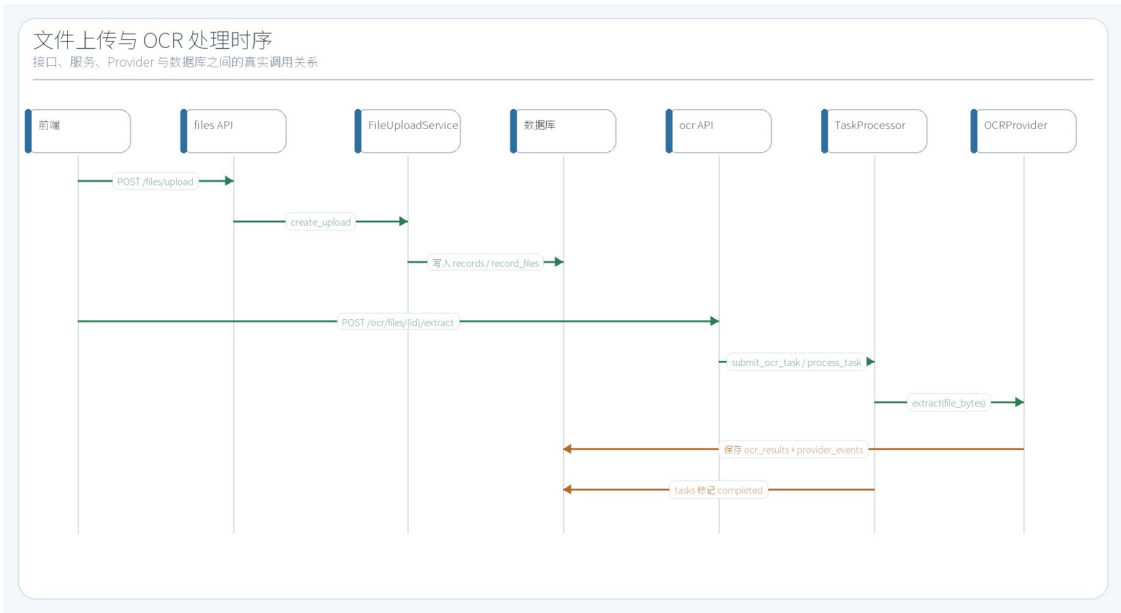


图 4-1 文件上传与 OCR 处理时序

4.2 标准化与版本化入库实现

标准化服务读取 `OCRResult.raw_text` 后调用 `NormalizationProvider`。当前实现为 `llm_direct`，并带规则兜底。`Provider` 输出包含 `document_type`、`document_category`、`report_date`、`measurements` 和 `prose_facts` 等字段。系统不会只保存模型输出文本，而是将结构化结果写入 `ExtractedDocument`、`DocumentVersion` 和 `Measurement`。这样既能支持前端文档展示，也能支持指标搜索和 LangGraph 审计。

标准化稳定性的关键在于 `schema` 约束、规则兜底和版本机制。LLM 输出容易受到提示词、输入格式和模型状态影响，因此系统需要将结果约束为可解析 `payload`；当 LLM 调用失败或字段缺失时，规则解析可以至少抽取日期、常见指标和原文事实；当同一文档重新标准化时，`DocumentVersion.snapshot_hash` 用于避免重复版本或定位差异。该设计降低了“标准化效果不稳定”对后续审计流程的影响。

图 4-2 展示标准化与版本化入库流程。OCR 原文进入 `NormalizationProvider`，输出 `NormalizationResult` 后形成 `ExtractedDocument` 当前投影、`DocumentVersion` 版本快照和 `Measurement` 指标索引。图中还标注了稳定性策略：LLM 失败时规则兜底，`snapshot_hash` 避免重复版本，`narrative_context` 不强行生成数值指标。

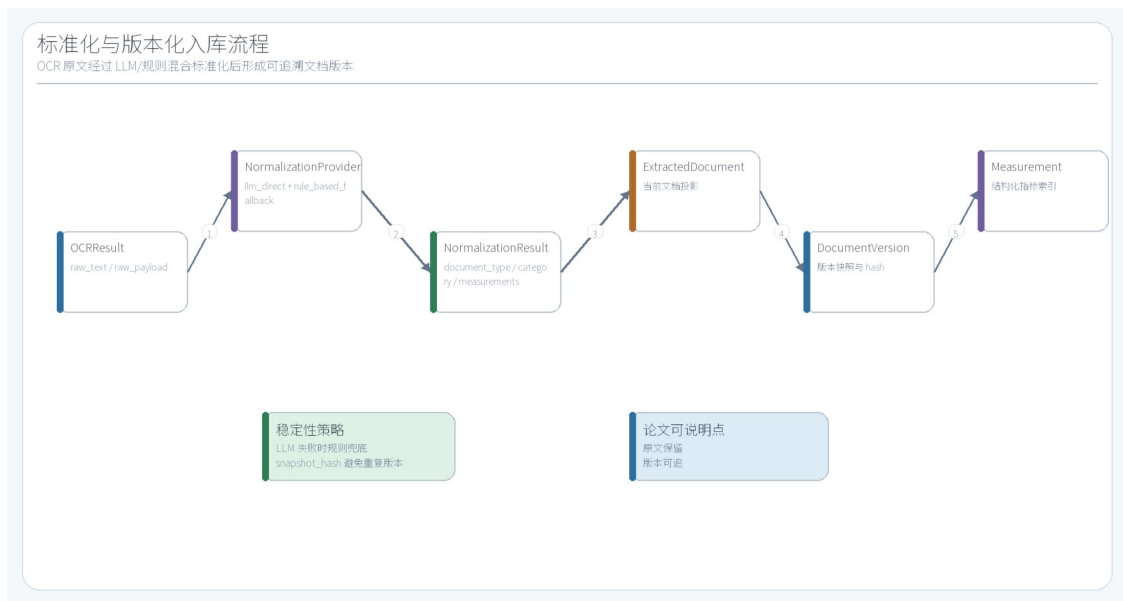


图 4-2 标准化与版本化入库流程

4.3 综合审计报告 LangGraph 状态机实现

表 4-1 LangGraph 节点职责与输入输出

节点	职责	输入依据	主要输出
load_graph_state	初始化状态	读取选中文档版本、指标、运行配置	next_action、 route_history
audit_router	主路由	根据 completed_agents 和 证据状态选择下一个审 计节点	next_action
document_quality_agent	文档质量审计	检查是否缺少文档、 OCR 原文、报告日期	quality_findings
timeline_builder	时间线构建	按报告日期和指标时间 生成时间线	timeline
measurement_consistency_agent	指标一致性审计	按规则检查血糖、 ALT、CRP、白细胞等异 常	consistency_findings
risk_agent	风险提示	将异常指标转化为非诊 断性关注项	risk_findings
evidence_agent	证据绑定	为风险、冲突和指标结 论绑定原文或指标证据	evidence_items
conflict_agent	冲突复核	检查叙事事实与结构化 指标是否存在复核点	conflict_findings
compliance_agent	合规检查	检查关键结论是否仍缺 少证据	compliance_findings

quality_gate	质量门禁	判断是否具备进入报告生成的最低条件	quality_gate
report_composer	报告生成	组织最终报告结构、结论和证据项	report_draft
citation_checker	引用检查	检查非 info 结论是否绑定证据	citation_issues
safety_reviewer	安全审查	拦截不适合审计场景的医疗表述	safety_issues
final_router	终态路由	根据引用和安全检查决定回退或持久化	next_action、stop_reason
persist_report	持久化	写入最终报告并结束图执行	final_report

综合审计报告模块是本文体现 LangGraph 架构的核心。服务层创建 AuditReportRun 后，把用户选择的 document_version_ids 转换为 AuditGraphState。状态中包含 documents、measurements、completed_agents、route_history、iteration_count、max_iterations、quality_findings、risk_findings、evidence_items、citation_issues、safety_issues 和 final_report 等字段。每个节点只返回局部更新，Engine 将更新合并回状态。

状态机的第一阶段由 audit_router 控制。audit_router 首先检查文档质量，再构建时间线，随后进行指标一致性审计、风险提示和证据绑定。当风险或冲突缺少证据时，_findings_need_evidence 会使路由回到 evidence_agent。普通审计节点完成后均回到 audit_router，这种回环不是前端动画，而是 LangGraph 中真实存在的条件边和状态迁移。

状态机的第二阶段由 final_router 控制。report_composer 生成报告草稿后，citation_checker 检查关键结论是否绑定证据，safety_reviewer 检查报告中是否包含不适合审计场景的医疗承诺。如果 citation_issues 存在，final_router 会回到 audit_router 补充审计；如果 safety_issues 存在，则回到 report_composer 重写报告；只有检查通过或达到最大迭代保护条件时，流程才进入 persist_report。

图 4-3 是综合审计报告 LangGraph 状态流转图，边关系来自 AUDIT_GRAPH_EDGES。图中蓝线表示 audit_router 分派审计节点，灰线表示节点完成后回到 audit_router，橙线表示报告生成与审查链路，红线表示 final_router 回环补充审计，绿线表示通过后持久化并结束。该图直接对应前端综合报告模块中需要展示的真实流转结构。

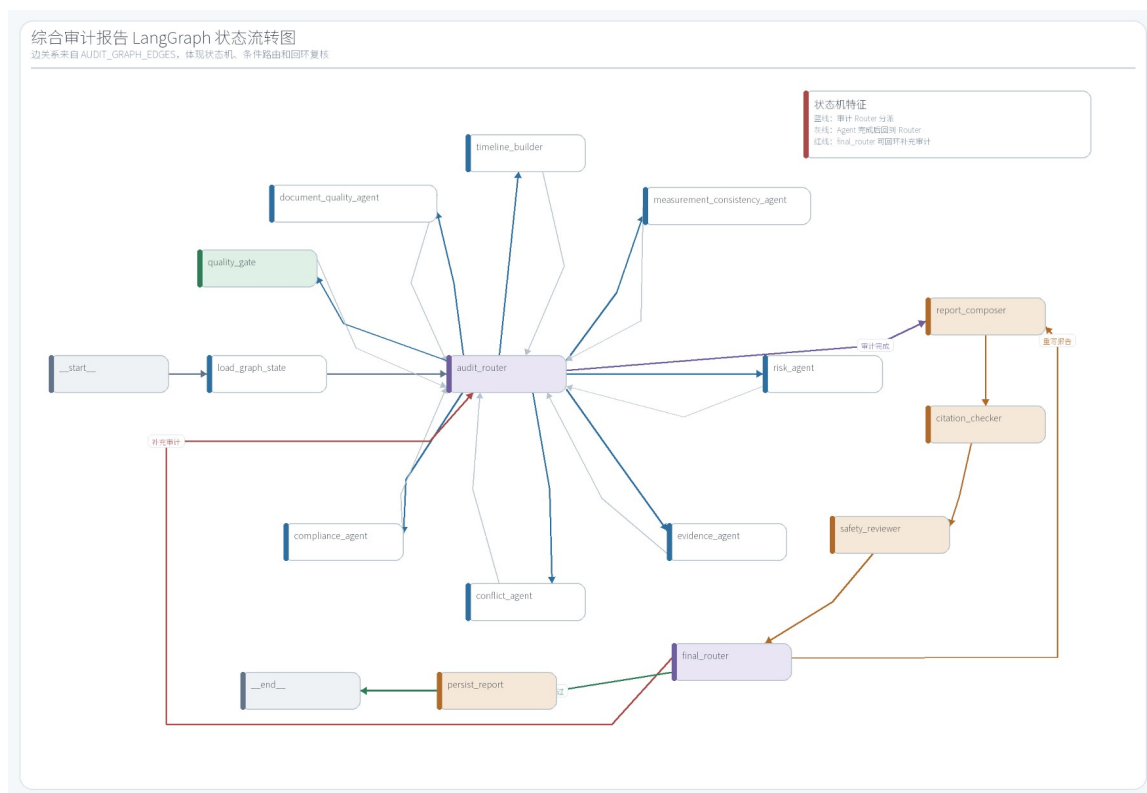


图 4-3 综合审计报告 LangGraph 状态流转图

4.4 审计事件与节点状态持久化

为了让前端看到“数据正在节点上流动”的真实效果，后端不能只在执行结束后返回报告，而需要在执行过程中记录事件。AuditGraphEngine.stream 每产生一个节点输出，服务层就保存一条 AuditReportEvent，并更新 AuditReportNodeState。事件包含 sequence、event_type、node_name、edge_source、edge_target、status、message 和 payload；节点状态包含 node_name、status、visit_count、last_event_id 和 output。

这种持久化方式使前端可以通过 GET /audit-reports/{run_id}/events 和 GET /audit-reports/{run_id}/nodes 轮询。前端不需要猜测流程走到哪里，而是根据真实事件高亮节点和边。若某个节点被回环再次访问，visit_count 会增加，route_history 也会体现重复流转。对于论文而言，这一点能够证明系统不是静态流程图，而是有后端状态支撑的 LangGraph 运行展示。

图 4-4 展示审计事件与节点状态持久化关系。AuditReportRun 保存整体运行状态和最终报告，AuditReportEvent 保存边和节点执行事件，AuditReportNodeState 保存每个节点的最新状态和输出，前端通过轮询接口展示节点高亮、边流转和完整报告按钮。

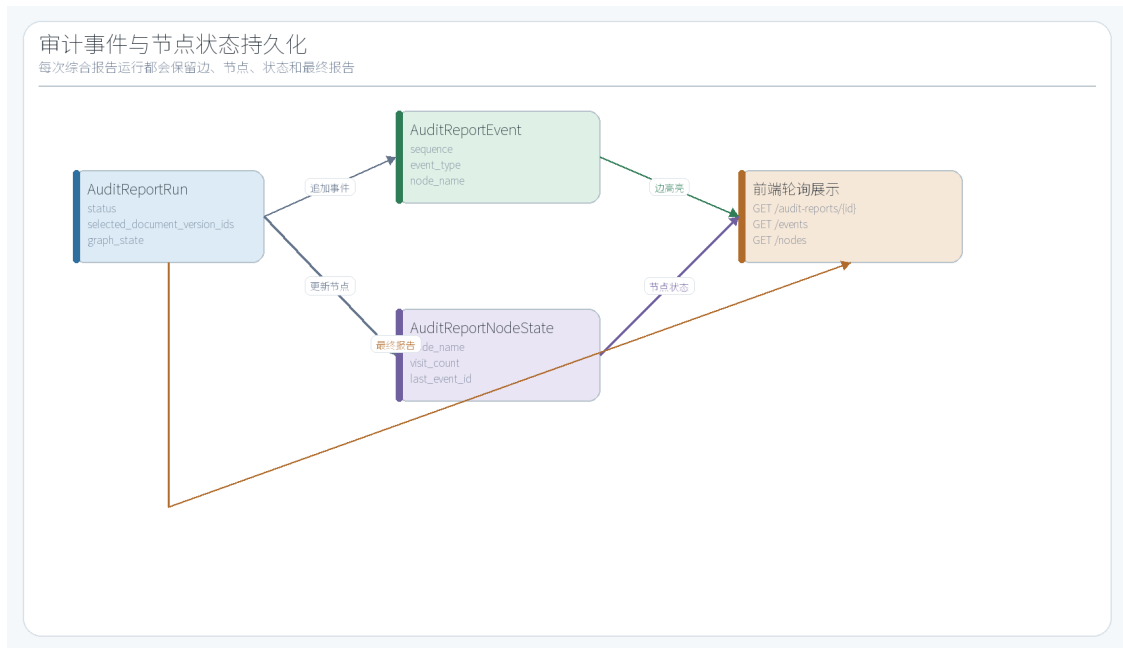


图 4-4 审计事件与节点状态持久化

4.5 前端模块实现

前端以工作台形式组织模块，主要包括系统控制台、文档接入流程、文档库、指标探索、智能洞察和综合审计报告。系统控制台用于查看服务状态和账号信息；文档接入流程负责上传和处理资料；文档库展示标准化文档与版本；指标探索支持结构化指标查询；智能洞察保持聊天机器人形态；综合审计报告模块独立承担文档选择、LangGraph 流程图流转和最终报告展示。

前端设计的重点是把智能洞察和综合审计报告分离。智能洞察是对话式能力，左侧选择数据源，中间为聊天框，右侧为对话记录；综合审计报告不是聊天框，而是一个以流程图为核心的工作台。用户选择若干文档后启动报告生成，流程图按照后端事件流实时高亮节点和边，最终通过按钮打开完整报告。这种区分可以避免所有能力挤在一个卡片中，也能突出 LangGraph 架构。

图 4-5 展示前端模块结构。该图来自 frontend/src/App.jsx 的模块配置和 API 封装设计，说明各模块共享 Token 和 API 服务，但在交互目标上保持分工。综合审计报告模块是论文中最能体现课题题目的前端入口。

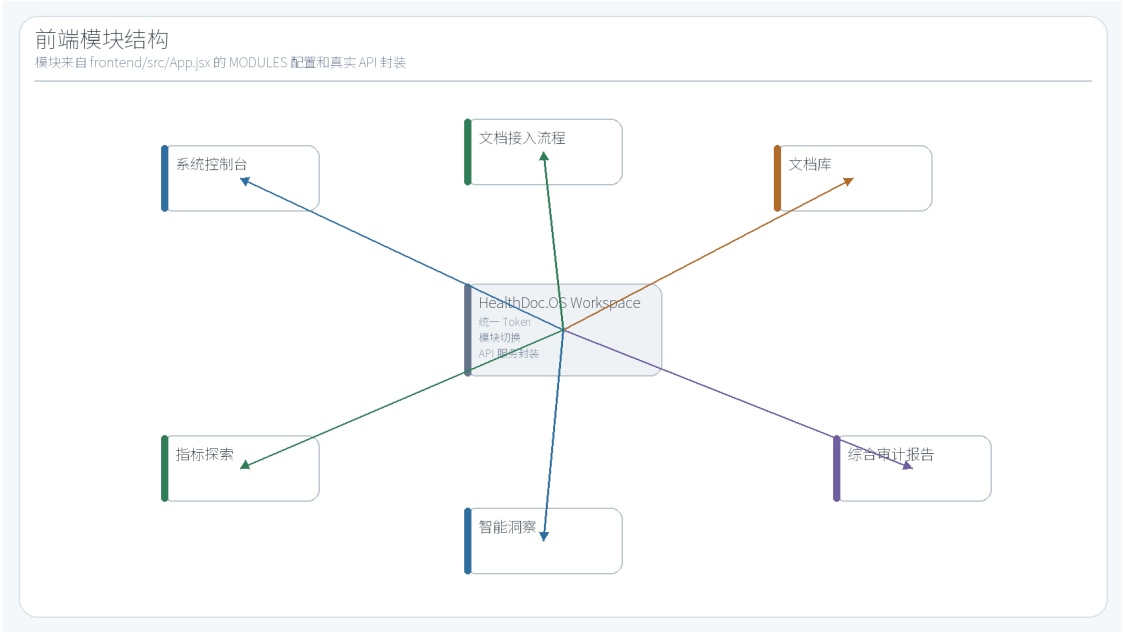


图 4-5 前端模块结构

第五章 测试与运行效果分析

本章基于 Mock 体检数据、核心接口测试和端到端运行链路说明系统运行效果，并对当前初稿阶段的完成度、可展示内容和后续完善方向进行分析。

5.1 Mock 体检数据构造

表 5-1 Mock 数据构成

项目	数量/分布	来源说明
文档总数	15	来自 scripts/seed_mock_exam_data. py 的 MOCK_DOCUMENTS
结构化指标	65	写入 measurements，用于指标 查询和审计
叙事实实	14	写入 normalized_payload.prose_fac ts
文档类型	physical_exam:2, lab_report:9, clinical_note:3, imaging_report:1	physical_exam、lab_report、 clinical_note、 imaging_report
月份分布	2026-01:3, 2026-02:2, 2026- 03:4, 2026-04:3, 2026-05:3	覆盖 2026 年 1 月至 5 月

为了支撑完整流程测试，项目提供 `seed_mock_exam_data.py` 脚本创建测试账户 `exam@healthdoc.local`，并写入一组模拟体检数据。该数据并不是随意生成的一两条记录，而是覆盖体检报告、化验报告、病历记录和影像结论等多种类型，使综合审计报告能够同时读取结构化指标和叙事事实。Mock 数据包括 15 份文档、65 条结构化指标和 14 条叙事事实。

Mock 数据的价值在于让系统能够稳定复现端到端流程。OCR Provider 在本地使用 `plaintext`，因此 Mock 文档以文本报告形式保存；标准化结果直接写入 `ExtractedDocument`、`DocumentVersion` 和 `Measurement`；综合审计报告可以选择多份文档运行 `LangGraph`。图 5-1 展示 Mock 体检数据分布，数据来源为脚本中的 `MOCK_DOCUMENTS` 定义。



图 5-1 Mock 体检数据分布

5.2 接口与端到端测试

表 5-2 端到端测试用例

编号	功能	接口/对象	预期结果	结果
T01	用户登录	POST /auth/login	返回 access_token	通过
T02	文件上传	POST /files/upload	创建 records 和 record_files	通过
T03	OCR 抽取	POST /ocr/files/{id}/extract	创建 ocr_results 和任务事件	通过
T04	标准化入库	POST /ingestion/ocr-results/{id}/normalize	创建文档、版本和指标	通过

T05	文档查询	GET /documents	返回用户文档列表	通过
T06	指标搜索	GET /measurements/search	返回匹配指标	通过
T07	综合审计创建	POST /audit-reports	创建 audit_report_runs	通过
T08	综合审计执行	POST /audit-reports/{run_id}/execute	产生事件、节点状态和最终报告	通过
T09	事件轮询	GET /audit-reports/{run_id}/events	返回边和节点执行事件	通过
T10	节点轮询	GET /audit-reports/{run_id}/nodes	返回节点状态和 visit_count	通过

测试围绕从登录到报告生成的主链路设计。首先验证用户认证是否能够返回 Token，然后验证文件上传、OCR、标准化、文档查询和指标查询，再验证综合审计报告的创建、执行、事件轮询和节点状态轮询。该测试路径覆盖了系统的核心业务数据流，也覆盖了论文中各图表对应的主要模块。

项目当前 pytest 用例已能够通过，临时后端健康检查接口返回 200。需要说明的是，毕业设计初稿阶段的测试仍以功能闭环验证为主，后续定稿前还应补充更多异常场景，例如空文档、缺失日期、标准化失败、LLM Provider 超时、审计报告安全审查不通过和最大迭代次数达到上限等。

图 5-2 展示端到端测试链路。该链路从登录和上传开始，经过 OCR、标准化、查询、审计运行、节点轮询和最终报告，能够对应用户演示时的完整操作路径。与纯后端单元测试相比，该链路更适合作为毕业答辩演示主线。



图 5-2 端到端测试链路

5.3 运行效果分析

从功能完成度看，当前项目已经具备后端演示和论文撰写的核心条件。资料上传、OCR 保存、标准化入库、指标查询、Mock 数据创建、LangGraph 综合审计报告、事件持久化和前端模块均已形成闭环。对于毕业设计而言，最重要的是能够展示“为什么必须使用 LangGraph”：系统中的审计流程包含多个审计节点、两个路由节点、状态字段、条件边和回环复核，而不是一条简单的线性流水线。

从技术体现看，综合审计报告模块解决了中期阶段“没有体现 LangGraph 架构”的问题。前端流程图高亮来自后端 `audit_report_events` 和 `audit_report_node_states`，后端事件来自 `AuditGraphEngine.stream`，边关系来自 `AUDIT_GRAPH_EDGES`。用户在页面上看到的不是静态 SVG，而是真实运行事件驱动的状态。该点应作为答辩演示的重点。

从不足看，当前系统仍有改进空间。第一，OCR Provider 在本地主要使用 `plaintext`，对图片型真实报告的识别能力需要接入真实 OCR 服务后再验证。第二，标准化效果虽然加入 `llm_direct` 和规则兜底，但仍需继续完善指标别名、单位换算和参考范围解析。第三，审计节点当前以规则审计为主，后续可以在安全边界明确的前提下引入更多 LLM 解释型节点。第四，数据库迁移链需要在定稿前整理，保证部署环境和本地环境一致。

结束语

本文围绕《基于 LangGraph 的多 Agent 协作框架在医疗审计场景的设计与实现》完成了系统需求分析、总体架构设计、数据库设计、接口设计、核心流程实现和测试说明。系统以个人健康资料为入口，完成文件上传、OCR、标准化入库、版本化存储、指标查询、综合审计报告和前端展示，形成了从原始资料到审计报告的工程闭环。

本文实现的关键特点在于使用 LangGraph 将综合审计报告生成过程组织为状态机。`audit_router` 负责分派文档质量、时间线、指标一致性、风险、证据、冲突、合规和质量门禁等节点；`final_router` 负责根据引用检查和安全审查决定回退或持久化；事件和节点状态被保

存到数据库并用于前端高亮展示。该设计使系统能够体现多 Agent 协作、条件路由、回环复核和可追溯执行过程。

后续工作主要包括三点。第一，接入更稳定的真实 OCR 服务和更多真实报告样本，提升图片、PDF 和复杂表格的处理能力。第二，完善标准化规则库和指标单位换算，增强不同报告模板之间的一致性。第三，在保持非诊断性边界的前提下扩展 LLM 审计节点，使风险解释、证据摘要和用户可读报告更加自然。总体来看，当前系统已经能够支撑毕业设计论文初稿和后续答辩演示，后续重点是补充测试、优化细节和完善论文表达。

参考文献

- [1] 肖仰华, 徐一丹. 大规模生成式语言模型在医疗领域的应用: 机遇与挑战[J]. 医学信息学杂志, 2023, 44(9):1-11.
- [2] 颜见智, 何雨鑫, 骆子烨, 等. 生成式大语言模型在医疗领域的潜在典型应用与面临的挑战[J]. 医学信息学杂志, 2023, 44(9):23-31.
- [3] 陈晓红, 刘浏, 袁依格, 等. 医疗大模型技术及应用发展研究[J]. 中国工程科学, 2024, 26(6):77-88.
- [4] 肖建力, 许东舟, 王浩, 等. 医疗领域的大型语言模型综述[J]. 智能系统学报, 2025, 20(3):530-547.
- [5] 国家卫生健康委员会规划与信息司. 全国医院信息化建设标准与规范(试行)[S/OL]. 2018-04.
- [6] 国家卫生健康委办公厅. 关于印发电子病历系统应用水平分级评价管理办法(试行)及评价标准(试行)的通知: 国卫办医函(2018)1079号[Z/OL]. 2018-12-03.
- [7] 国家市场监督管理总局, 国家标准化管理委员会. 信息安全技术 个人信息安全规范: GB/T 35273-2020[S/OL]. 2020-03-06.
- [8] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]//Advances in Neural Information Processing Systems. 2020, 33:9459-9474.
- [9] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C]//International Conference on Learning Representations. 2023.
- [10] Wu Q, Bansal G, Zhang J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation[EB/OL]. arXiv:2308.08155, 2023.
- [11] Thirunavukarasu A J, Ting D S J, Elangovan K, et al. Large Language Models in Medicine[J]. Nature Medicine, 2023, 29:1930-1940.
- [12] Singhal K, Azizi S, Tu T, et al. Large Language Models Encode Clinical Knowledge[J]. Nature, 2023, 620:172-180.
- [13] LangChain. LangGraph overview[EB/OL]. <https://docs.langchain.com/oss/python/langgraph/overview>, 2026-05-07.
- [14] FastAPI. FastAPI documentation[EB/OL]. <https://fastapi.tiangolo.com/>, 2026-05-07.

- [15] SQLAlchemy. SQLAlchemy 2.0 Documentation[EB/OL].
<https://docs.sqlalchemy.org/20/>, 2026-05-07.
- [16] Pydantic. Pydantic documentation[EB/OL]. <https://docs.pydantic.dev/>, 2026-05-07.